

Pet Match AI

Flutter · Android-first · Architecture & Implementation Plan

Описание архитектуры, принятых решений и компромиссов

Cubit · go_router · dio · get_it · Clean Architecture

Стек	Flutter 3.x · Dart 3.x · Android-first
State management	flutter_bloc (Cubit)
Навигация	go_router
HTTP	dio + interceptors
DI	get_it + injectable
Локаль API	ru (?locale=ru)

- 1.** Анализ задания
- 2.** Изучение API — ключевые находки
- 3.** Обоснование выбора технологий
- 4.** Архитектура — Clean Architecture
- 5.** Структура проекта
- 6.** Data Layer
- 7.** Domain Layer
- 8.** Presentation Layer
- 9.** Навигация
- 10.** Обработка ошибок
- 11.** Кеширование и продолжение сессии
- 12.** Тесты
- 13.** Компромиссы и дальнейшие улучшения

Анализ задания

Задание — реализовать Android-first Flutter-приложение, воспроизводящее сценарий анкетирования сервиса Pet Match AI. Цель не в полноте функциональности, а в качестве кода, архитектурных решениях и правильной обработке состояний и ошибок.

Что оценивается

- **Архитектура и организация кода** — чёткое разделение на слои, отсутствие бизнес-логики в UI
- **Async и state handling** — все состояния явные, правильный жизненный цикл Cubit
- **UI-реализация** — адаптивность, аккуратность, соответствие продуктовому UX
- **Обработка ошибок** — каждый тип ошибки обрабатывается отдельно и осмысленно
- **Читаемость кода** — понятные имена, комментарии там, где неочевидно

Обязательные экраны

Экран	Назначение	API
Welcome	Стартовый экран, генерация uid	—
Intro	Вводный экран, запуск анкеты	—
Questionnaire	Вопрос, прогресс, отправка ответа	POST /start, POST /answers
Analyzing	Ожидание результата, polling	GET /session (polling)
Result	Основная порода + альтернативы	из сессии после polling
Breed Details	Секции описания породы	GET /breeds/{id}
Breed Gallery	Фотогалерея породы	gallery_images[] из breed detail

Изучение API — ключевые находки

Перед проектированием был изучен OpenAPI-контракт сервиса. Ниже — всё, что влияет на архитектуру и принятые решения.

Эндпоинты и их роль

POST /questionnaire/start	Создаёт или возобновляет сессию. Принимает external_id. Возвращает QuestionnaireSessionRead — центральный объект со всем состоянием: данные пользователя, статистика, следующий вопрос, результат совместимости.
POST /users/{id}/answers	Отправляет ответ на текущий вопрос. Возвращает обновлённую сессию с новым next_question. Если next_question == null — анкета завершена.
GET /users/{id}/session	Возвращает текущее состояние сессии. Используется для polling на экране Analyzing — опрашиваем каждые 1.5 секунды пока compatibility.status не сигнализирует о готовности результата.
GET /users/{id}/questions/{q_id}/options	Динамические опции для вопроса — например, список пород по выбранному типу питомца. Поддерживает текстовый поиск (?q=) и лимит результатов.
POST /users/{id}/questions/{q_id}/skip	Пропуск необязательного вопроса. Возвращает обновлённую сессию.
GET /breeds/{breed_id}	Детали породы: название, описание, изображение, массив фотографий для галереи, секции с подробным текстом (характер, уход, особенности).

Ключевые особенности API, влияющие на архитектуру

- **Единый объект сессии** — QuestionnaireSessionRead содержит всё: пользователя, статистику прогресса, текущий вопрос и результат. Это упрощает state management — один объект, одно состояние.
- **Завершение анкеты через null** — Когда next_question == null — анкета завершена. Не нужен отдельный флаг или эндпоинт. Проверяем после каждого ответа.
- **Три типа вопросов** — single_choice (выбор одного варианта), multiple_choice (несколько вариантов), dynamic_options (варианты подгружаются с сервера с поддержкой поиска). Каждый тип требует своего UI-компонента.
- **Polling для результата** — После завершения анкеты бэкенд считает совместимость асинхронно. Клиент опрашивает GET /session каждые 1.5 секунды. Максимальное время ожидания — 30 секунд, после чего показываем ошибку.
- **Идентификация пользователя** — external_id формируется как uid:. UUID генерируется на клиенте при первом запуске и сохраняется. Один и тот же UUID =

продолжение сессии. Новый UUID = новая анкета с нуля.

- **Альтернативные рекомендации** — Объект совместимости содержит массив suggestions — альтернативные породы со своими оценками. Отображаются на экране результата ниже основной рекомендации.

Обоснование выбора технологий

flutter_bloc — Cubit

State management

Cubit выбран вместо полноценного Bloc (с Events), потому что flow приложения линейный: нет сложных реактивных цепочек, нет конкурентной обработки нескольких потоков событий. Cubit — это Bloc без Events: вся логика в явных методах, состояния — sealed-классы. Результат: меньше бойлерплейта, проще читать на ревью, легче тестировать.

go_router

Навигация

Официально рекомендованный пакет от Flutter team. Поддерживает declarative routing, redirect guards для проверки наличия сессии, deep links, типизированную передачу данных. Navigator 2.0 под капотом без его сложности. Альтернатива auto_route сложнее в настройке без существенных преимуществ для данного проекта.

dio

HTTP клиент

Поддерживает Interceptors из коробки — это принципиально важно для retry-логики, логирования и автоматического добавления параметра локали (?locale=ru) к каждому запросу. Встроенный http пакет interceptors не поддерживает. Настроены три interceptora: RetryInterceptor, LocaleInterceptor, LoggingInterceptor.

get_it + injectable

Dependency Injection

get_it — service locator, injectable генерирует регистрации через аннотации (@injectable, @singleton). Cubit регистрируется как factory — новый экземпляр при каждом создании. Репозитории и DataSource — как singleton. Инициализация один раз в main.dart, никакой зависимости от BuildContext в нижних слоях.

shared_preferences

Локальное хранилище

Используется для хранения двух значений: uid (идентификатор анкеты) и user_id (ID из API). Этого достаточно для продолжения сессии после перезапуска приложения. Hive и Drift избыточны для этой задачи и увеличат score без пользы.

cached_network_image

Загрузка изображений

Кеширует изображения пород на диск. Показывает плейсхолдер во время загрузки и виджет-заглушку при ошибке. Необходим для галереи, где одновременно загружается много фотографий.

От чего отказались и почему

- **Bloc с Events** — Избыточен для линейного flow без сложных реактивных цепочек. Cubit делает то же самое проще и читаемее.
- **Riverpod** — Мощнее и декларативнее, но выше порог вхождения. Cubit + go_router полностью закрывают потребности проекта.
- **Provider** — Устаревший подход, flutter_bloc активнее поддерживается и лучше документирован.
- **Hive / Drift** — Избыточно для хранения двух примитивных значений. SharedPreferences достаточно.

Архитектура — Clean Architecture

Проект строится на Clean Architecture с тремя слоями: data, domain, presentation. Направление зависимостей строго одностороннее: presentation зависит от domain, data реализует интерфейсы domain. Domain не знает ни о Flutter, ни о конкретных HTTP-библиотеках — только чистые Dart-классы.

presentation/

Screens · Cubits · Widgets · Router

Самый верхний слой. Отвечает только за отображение состояний и передачу пользовательских действий в Cubit. Cubit вызывает use case, получает результат или ошибку, эмитит новое состояние. Никакого dio, DTO или бизнес-логики здесь. Знает только о domain entities.

domain/

Entities · UseCases · Repository interfaces · Failures

Ядро приложения. Чистый Dart, никаких Flutter-зависимостей. Repository определён как абстрактный интерфейс — domain не знает, откуда приходят данные. Use case — один класс, один публичный метод call(), одна ответственность. AppFailure — sealed class для типизированных ошибок вместо строк или Exception.

data/

DTOs · Mappers · RemoteDataSource · Repository impl

Реализует интерфейсы из domain. DTO — прямое отражение JSON API, генерируется через json_serializable + freezed. Mapper конвертирует DTO в Entity. RemoteDataSource содержит только сырые HTTP-вызовы. Repository перехватывает DioException и конвертирует в AppFailure.

core/

DI · Network config · Theme · Cache

Сквозная инфраструктура: не бизнес-логика и не UI. Конфигурация dio с interceptors, регистрация зависимостей get_it, обёртка SharedPreferences, тема приложения.

Правило зависимостей

presentation → domain ← data. data реализует интерфейсы domain, но domain не импортирует data. Это означает, что реализацию репозитория можно заменить (например, на mock для тестов) без изменения domain или presentation слоёв.

Структура проекта

Каждая папка соответствует одному слою или одной функциональной области. Внутри `presentation/` каждый экран живёт в своей папке вместе со своим Cubit. Тесты в `test/` зеркально повторяют структуру `lib/`.

core/

`di/` – регистрация зависимостей `get_it` + `injectable`
`network/` – конфигурация `dio`, `interceptors`, `ApiException`
`cache/` – `SessionCache`, обёртка над `SharedPreferences`
`theme/` – цветовая схема и типографика

data/

`dto/` – DTO-классы (`freezed` + `json_serializable`)
`mappers/` – конвертация DTO → Entity
`sources/` – `QuestionnaireRemoteSource` (сырые HTTP-вызовы)
`repositories/` – реализации интерфейсов из `domain`

domain/

`entities/` – `Question` (`sealed`), `Session`, `Compatibility`, `BreedDetail`
`failures/` – `AppFailure` (`sealed`): `Network`, `Timeout`, `Server`, `Empty`
`repositories/` – абстрактные интерфейсы `IQuestionnaireRepository`, `IBreedRepository`
`use_cases/` – `StartSession`, `SubmitAnswer`, `SkipQuestion`, `GetDynamicOptions`, `PollCompatibility`, `GetBreedDetail`

presentation/

`router/` – `AppRouter` (`go_router` + `redirect guards`)
`welcome/` – `WelcomePage`
`intro/` – `IntroPage`
`questionnaire/` – `QuestionnairePage` + `QuestionnaireCubit`
`widgets/` – `SingleChoiceWidget`, `MultiChoiceWidget`, `DynamicOptionsWidget`, `ProgressBarWidget`
`analyzing/` – `AnalyzingPage` (`polling` + анимация ожидания)
`result/` – `ResultPage` + `ResultCubit`
`widgets/` – `MainBreedCard`, `SuggestionCard`
`details/` – `BreedDetailPage`, `BreedGalleryPage` + `BreedDetailCubit`

Data Layer

DTO — Data Transfer Objects

DTO — это прямое отражение JSON-контракта API. Генерируются через связку `json_serializable` + `freezed`. `Freezed` автоматически даёт `immutability`, метод `copyWith` для создания модифицированных копий и правильную реализацию `equality` (сравнение по значению, а не по ссылке). Ни один DTO не попадает выше `data` слоя.

Mappers

`Mapper` — статический класс с методом `fromDto()`. Это чистая функция без зависимостей: принимает DTO, возвращает `Entity`. Вся логика преобразования сосредоточена здесь: обработка `nullable` полей, конвертация строковых типов в `sealed`-классы, присваивание дефолтных значений.

Критически важный случай — `QuestionMapper`: принимает DTO с полем `question_type: String` и возвращает один из трёх `sealed`-вариантов (`SingleChoiceQuestion`, `MultipleChoiceQuestion`, `DynamicOptionsQuestion`). Неизвестный тип даёт `fallback` на `SingleChoiceQuestion`, чтобы новые типы вопросов не роняли приложение.

RemoteDataSource

Содержит только сырые HTTP-вызовы через `dio`. Не знает о бизнес-логике, не обрабатывает ошибки — просто выполняет запрос и возвращает DTO. При сетевой ошибке пробрасывает `DioException` наверх.

Repository Implementation

Мост между `data` и `domain`. Вызывает `RemoteDataSource`, прогоняет результат через `Mapper`, возвращает `Entity`. Перехватывает `DioException` и конвертирует в типизированный `AppFailure`: `connectionError` → `NetworkFailure`, `timeout` → `TimeoutFailure`, HTTP-ошибки → `ServerFailure` с кодом статуса. Здесь же живёт `in-memory` кеш для `BreedDetail`.

Domain Layer

Sealed class Question

Вместо проверки строкового поля `question_type` в runtime используется `sealed class`. Компилятор Dart гарантирует exhaustive обработку всех вариантов в `switch` — если добавить новый тип вопроса и забыть обработать его в UI, получим ошибку компиляции, а не падение в production. Три конкретных класса: `SingleChoiceQuestion`, `MultipleChoiceQuestion`, `DynamicOptionsQuestion`.

Sealed class AppFailure

Типизированные ошибки вместо `Exception` или строк. Четыре варианта: `NetworkFailure` (нет подключения), `TimeoutFailure` (превышено время), `ServerFailure` (HTTP-ошибка с кодом), `EmptyResponseFailure` (пустой ответ). В UI каждый вариант обрабатывается отдельно — своё сообщение пользователю.

Use Cases

Каждый use case — один класс с одним публичным методом `call()`. Это принцип единственной ответственности: один класс делает одно дело. Зависимости передаются через конструктор и инжектируются через `get_it`.

Список use cases:

- **StartSession** — Генерирует или читает `uid` из кеша, вызывает `POST /start`, сохраняет полученный `user_id`. Возвращает `Session`.
- **SubmitAnswer** — Принимает ответ пользователя в типизированном формате (`option_id`, `option_ids` или `selected_value`), отправляет `POST /answers`, возвращает обновлённую `Session`.
- **SkipQuestion** — Отправляет `POST /skip` для необязательного вопроса. Возвращает обновлённую `Session`.
- **GetDynamicOptions** — Вызывает `GET /options` с поисковым запросом. Используется в `DynamicOptionsWidget` с `debounce 400` мс.
- **PollCompatibility** — Циклически опрашивает `GET /session` каждые 1.5 секунды пока `compatibility.status` не сигнализирует готовность. Бросает `TimeoutFailure` через 30 секунд.
- **GetBreedDetail** — Вызывает `GET /breeds/{id}`. Проверяет in-memory кеш перед сетевым запросом.

Presentation Layer

Структура состояний Cubit

Каждый Cubit имеет sealed class состояний. Это гарантирует exhaustive обработку в BlocBuilder — нельзя случайно пропустить состояние. Состояния immutable, содержат только то, что нужно для рендеринга.

QuestionnaireCubit управляет следующими состояниями:

- **QuestionnaireInitial** — начальное состояние до старта
- **QuestionnaireLoading** — загрузка сессии или отправка ответа
- **QuestionnaireQuestion** — текущий вопрос + данные прогресса (percent, answered, total)
- **QuestionnaireCompleted** — анкета завершена, next_question == null
- **QuestionnaireError** — ошибка с типизированным AppFailure

Экраны и их Cubit

QuestionnairePage

Cubit: QuestionnaireCubit Центральный экран. BlocConsumer: builder рендерит виджет по состоянию (loading, question, error), listener реагирует на QuestionnaireCompleted — переходит на /analyzing. Вся логика в Cubit, виджет только отображает и передаёт действия.

AnalyzingPage

Cubit: — (StatefulWidget) Запускает PollCompatibility в initState. Показывает анимацию ожидания. При успехе переходит на /result, при TimeoutFailure показывает ошибку с возможностью вернуться и начать заново.

ResultPage

Cubit: ResultCubit Отображает основную рекомендацию породы (breed_name, score, risk_level, summary, insights) и список альтернатив из suggestions[]. Каждая альтернатива — карточка с переходом на детальный экран.

BreedDetailPage

Cubit: BreedDetailCubit Загружает детали породы через GetBreedDetail. Рендерит секции (sections[]) как список карточек с заголовком и текстом. Кнопка открытия галереи — если gallery_images[] не пустой.

BreedGalleryPage

Cubit: — (StatelessWidget) GridView с фотографиями через `cached_network_image`.
Данные передаются через `extra` в `go_router` — отдельный запрос не нужен.

DynamicOptionsWidget

Вопросы типа `dynamic_options` требуют подгрузки вариантов с сервера. Виджет содержит текстовое поле для поиска. `Debounce 400` мс исключает запрос на каждый нажатый символ. Опции хранятся в локальном `ValueNotifier` — отдельный Cubit избыточен для этого простого состояния.

Навигация — go_router

go_router позволяет объявить все маршруты декларативно в одном месте. Redirect guard проверяет наличие сохранённого user_id при старте: если сессия есть — пропускаем Welcome и Intro, переходим сразу на Questionnaire для продолжения анкеты.

Маршруты

/	redirect → /welcome
/welcome	WelcomePage
/intro	IntroPage
/questionnaire	QuestionnairePage
/analyzing	AnalyzingPage (extra: userId: int)
/result	ResultPage (extra: Compatibility)
/breed/:id	BreedDetailPage (pathParam: breedId)
/breed/:id/gallery	BreedGalleryPage (extra: List images)

Данные между экранами передаются через параметр extra. Это позволяет избежать глобального состояния: ResultPage получает готовый объект Compatibility напрямую, не нужен никакой SharedState или InheritedWidget.

Обработка ошибок и состояний

Каждый тип ошибки обрабатывается отдельно и показывает пользователю осмысленное сообщение. Retry доступен на каждом экране.

NetworkFailure

Нет подключения к интернету

Сообщение: "Нет подключения к сети. Проверьте интернет и попробуйте снова."
Кнопка Retry повторяет последний запрос.

TimeoutFailure

Превышено время ожидания запроса или polling

Сообщение: "Сервер долго не отвечает. Попробуйте позже." На экране Analyzing — дополнительное предложение вернуться и начать анкету заново.

ServerFailure

HTTP 4xx/5xx от сервера

Код статуса логируется для отладки. Пользователю: "Что-то пошло не так. Попробуйте снова." Retry доступен.

EmptyResponseFailure

Пустой или неожиданный формат ответа

Трактуется как ошибка сервера. Пользователю: "Нет данных. Попробуйте снова." Retry доступен.

RetryInterceptor

dio Interceptor автоматически повторяет запрос при NetworkFailure и TimeoutFailure. Максимум 3 попытки. Задержка между попытками — exponential backoff: 1 секунда перед второй попыткой, 2 секунды перед третьей, 4 перед четвёртой. Это снижает нагрузку на сервер и даёт время на восстановление соединения. Server errors (4xx/5xx) не ретраются автоматически.

AppErrorWidget

Переиспользуемый виджет принимает AppFailure и onRetry callback. По типу failure выбирает нужное сообщение через switch (exhaustive, sealed class). Показывает иконку, текст и кнопку. Используется на всех экранах — единая точка для изменения внешнего вида ошибок.

Кеширование и продолжение сессии

SessionCache — SharedPreferences

Персистируем два значения: `uid` и `user_id`. При запуске приложения проверяем наличие сохранённого `user_id`: если есть — вызываем `GET /session` для продолжения, если нет — генерируем новый `uid` и вызываем `POST /start` для создания новой сессии. Это реализует требование "базового кеширования для продолжения сессии" из задания.

Метод `clearSession()` используется на `Welcome`-экране, если пользователь явно хочет начать анкету заново.

In-memory кеш для Breed Detail

`BreedRepositoryImpl` хранит загруженные детали пород в `Map`. Перед сетевым запросом проверяет кеш. Это исключает повторные запросы при навигации: пользователь открыл детали породы, вернулся к результату, снова открыл детали — сетевого запроса не будет. Данные не меняются в рамках сессии, персистировать их на диск нет смысла.

Тесты

Задание требует минимум 2-3 теста. Реализовано 5: 3 unit-теста и 2 widget-теста. Каждый проверяет реальное поведение, а не тривиальные геттеры.

#1 Unit

QuestionMapperTest

QuestionMapper корректно конвертирует DTO в sealed entity. Проверяемые случаи: single_choice → SingleChoiceQuestion с правильными опциями, multiple_choice → MultipleChoiceQuestion, неизвестный тип → fallback на SingleChoiceQuestion, null-опции не вызывают NullPointerException.

#2 Unit

QuestionnaireRepositoryTest

При DioExceptionType.connectionError репозиторий бросает NetworkFailure. При connectionTimeout — TimeoutFailure. При HTTP 422 — ServerFailure с правильным statusCode. RemoteDataSource заменён Mockito-моком — сетевых запросов нет.

#3 Unit

PollCompatibilityTest

Use case возвращает Compatibility когда compatibility.isReady == true. Бросает TimeoutFailure если истекли 30 секунд без готового результата. Используется FakeClock — никакого реального ожидания в тесте.

#4 Widget

QuestionnairePageTest

Когда Cubit эмитит QuestionnaireLoading — на экране виден CircularProgressIndicator и нет кнопок ответа. Когда эмитит QuestionnaireError(NetworkFailure) — виден текст "Нет подключения" и кнопка Retry.

#5 Widget

ErrorWidgetRetryTest

AppErrorWidget с NetworkFailure и onRetry callback: при нажатии Retry callback вызывается ровно один раз. Текст сообщения соответствует ожидаемому для NetworkFailure.

Компромиссы и дальнейшие улучшения

Намеренно упрощено

- **Локализация UI** — Строки интерфейса захардкожены на русском языке. API сам возвращает тексты вопросов, вариантов и описаний с `?locale=ru`, поэтому основной контент локализован. ARB-файлы и пакет `intl` не добавлялись — это дополнительные часы без прироста в оценке архитектуры.
- **Анимации переходов** — Используются стандартные переходы `go_router`. Нет кастомных `slide`- или `fade`-анимаций между вопросами. Не критично для оценки архитектурных решений.
- **Галерея** — Реализован `GridView` с `cached_network_image`. Нет `hero`-анимаций при открытии, `pinch-to-zoom` и полноэкранного просмотра.
- **Offline-режим** — Только `in-memory` кеш для `breed detail`. Нет сохранения прогресса анкеты на диск и синхронизации при восстановлении сети.
- **Аналитика** — `POST /events (result_viewed, alternative_viewed)` не реализован — задание явно не требует этого эндпоинта.
- **Accessibility** — Базовые semantics Flutter. Нет проработанных screen reader labels и кастомного порядка фокуса для TalkBack.

Следующие шаги при большем времени

- **Персистентный кеш сессии** — `Hive` или `Drift` для хранения прогресса анкеты на диске. Полноценный `offline-first`: пользователь может отвечать без сети, ответы синхронизируются при восстановлении подключения.
- **Анимации** — `Lottie` для экрана `Analyzing` (анимация ожидания), `AnimatedSwitcher` между вопросами, `Hero`-анимации для изображений пород.
- **Golden tests** — `Snapshot`-тесты для ключевых виджетов: `BreedCard`, `QuestionCard`, `AppErrorWidget`. Защищают от случайных визуальных регрессий.
- **CI/CD** — `GitHub Actions`: `flutter analyze`, `flutter test`, `flutter build apk` на каждый PR. APK как артефакт релиза.
- **Accessibility** — `Semantics` для всех интерактивных элементов, проработанный порядок фокуса, полная поддержка TalkBack.
- **POST /events** — Аналитика просмотров результата и альтернатив согласно API-контракту.

Итог

Проект построен на Clean Architecture с чётким разделением слоёв. Ключевые решения: sealed Question для типобезопасной обработки типов вопросов, sealed AppFailure для типизированных ошибок, Cubit для явного управления состоянием, go_router для декларативной навигации с redirect guards. Каждый тип ошибки обрабатывается осмысленно с возможностью retry. Это именно то, что оценивает команда в первую очередь.